

Trained to React: Gesture-Triggered Sound with micro:bit

Grade Band: 9-12

Design a motion-recognition system that turns micro:bit gestures into programmed sound outputs. Collect accelerometer data, train and evaluate a CreateAI model, build a MakeCode response, and investigate the reliability of an embedded AI interaction.

Equipment, Curriculum, and Training

7 Lesson Hours; Equipment: micro:bit, USB cable, battery pack (optional); Software: micro:bit CreateAI; Curriculum and supporting materials; ongoing product and curriculum support; professional development; facilitation by a trained STEM instructor (optional).

Lesson 1: Connect the micro:bit and Explore Motion Input

Learning Target: Connect a compatible micro:bit to CreateAI, observe how movement changes accelerometer readings, and distinguish motion input from sound output.

Introduces: checking micro:bit V2 compatibility and locating its built-in speaker; connecting micro:bit via USB; opening micro:bit CreateAI in a supported browser; installing CreateAI connection software on micro:bit and pairing it with the browser; reading x/y/z accelerometer data; distinguishing accelerometer input from programmed sound output

Content

Today's focus is the difference between what a device senses and what it produces.

Motion in, sound out

micro:bit CreateAI trains machine-learning models using accelerometer/motion data, not audio or images. It can trigger programmed sound output through MakeCode in response to recognized motion. In this project, movement is the input and a programmed sound is the output—not the other way around.

An accelerometer measures acceleration along three axes, labeled x, y, and z. Its readings include the effect of gravity, so a stationary device does not necessarily have three zero readings. Rotating the device changes how gravity is distributed across its sensing axes. Moving it can also change its readings.

A micro:bit V2 has a built-in speaker on its back. For this course, check for the V2 board and locate that speaker, but do not program a sound yet.

Setup boundary

The supplied CreateAI capability description does not establish browser requirements, connection-software installation, pairing procedures, or a live accelerometer display. Accordingly, leave those procedures pending rather than inventing setup steps. This lesson's supported activity covers hardware preparation, physical exploration, and interpretation of illustrative readings. Do not report a completed CreateAI connection or observations of live readings. Completing those parts of the learning target requires verified setup instructions and confirmation of the relevant tool capabilities.

Activity

1. Inspect the hardware.

Locate the micro:bit's version marking and check whether it identifies a V2 board. Locate the built-in speaker on the back. If the version is unclear or the speaker cannot be identified, ask the teacher to verify the board before proceeding.

2. Prepare—but do not claim to complete—the connection.

Find the board's USB socket and the matching end of the supplied cable. Gently insert that end without forcing it. Leave the other end unconnected: a USB host/access device is not specified in the equipment list. Mark your setup status as "cable attached to board; powered connection and CreateAI pairing pending." Leave the optional battery pack disconnected.

3. Explore orientation safely.

With the board supported close to the work surface and without pulling the cable, hold it still with its front facing upward. Slowly turn it onto an edge, then return it to its original orientation. Describe what you physically changed. Do not describe these movements as measured data: no live readings have been obtained.

4. Interpret example readings.

Use this illustrative table. These are simplified readings from a generic accelerometer, not measurements collected from your micro:bit. The unit g represents acceleration relative to Earth's gravity.

Position A, stationary: $x = 0$ g; $y = 0$ g; $z = 1$ g.

Position B, stationary after rotation: $x = 0$ g; $y = 1$ g; $z = 0$ g.

Position A again, stationary: $x = 0$ g; $y = 0$ g; $z = 1$ g.

Discuss: Which axes changed between A and B? Why can a stationary sensor have a nonzero reading? What pattern returns when the sensor returns to A? Do not assume these example positions identify the axes on your particular board.

5. Trace the direction of information.

Complete this statement aloud: "The input for our project is _____. The eventual output is _____." Then explain why making a noise near the device is not an example of the input used to train this project.

6. Give a status report.

Report three things: the board version you identified, one interpretation of the example readings, and which setup tasks remain pending. Stop before defining gesture classes, collecting labeled examples, training a model, or programming sounds.

Equipment Used: micro:bit, USB cable

Claims (10) — review these first

[HARD_FACT] (source: micro:bit Educational Foundation — Official CreateAI product page and user guide; motion-data training, as specified in the supplied capabilities summary.)

micro:bit CreateAI trains machine-learning models using accelerometer/motion data, not audio or images.

[HARD_FACT] (source: micro:bit Educational Foundation — Official CreateAI product page and user guide; programmed sound responses, as specified in the supplied capabilities summary.)

It can trigger programmed sound output through MakeCode in response to recognized motion.

[MECHANISTIC] In this project, movement is the input and a programmed sound is the output—not the other way around.

[MECHANISTIC] An accelerometer measures acceleration along three axes, labeled x, y, and z.

[CAUSAL] Its readings include the effect of gravity, so a stationary device does not necessarily have three zero readings.

[MECHANISTIC] Rotating the device changes how gravity is distributed across its sensing axes.

[MECHANISTIC] Moving it can also change its readings.

[HARD_FACT] (source: micro:bit Educational Foundation — micro:bit overview: board features and rear-view identification of the built-in speaker on V2.) A micro:bit V2 has a built-in speaker on its back.

[HARD_FACT] (source: Supplied course input — third_party_software entry for micro:bit CreateAI, capabilities_summary; these setup and display capabilities are not specified.) The supplied CreateAI capability description does not establish browser requirements, connection-software installation, pairing procedures, or a live accelerometer display.

[MECHANISTIC] The unit g represents acceleration relative to Earth's gravity.

Assessment

For this project, which description correctly distinguishes input from output?

- A spoken word is the input, and motion is the output.
- Movement measured by the accelerometer is the input, and a programmed sound is the output. ✓
CORRECT
- A camera image is the input, and acceleration is the output.
- A programmed sound is both the training input and the output.

(Correct answer justified by claim #2 above)

Lesson 2: Define Gestures and a No-Action Class

Learning Target: Define two distinguishable gestures and an idle/background class, then plan labeled recordings with consistent device placement and varied repetitions.

Introduces: defining gesture classes and an idle/background class; using class labels for supervised learning; controlling micro:bit placement and orientation; planning varied and balanced motion recordings; reserving fresh gesture performances for evaluation

Prerequisites: reading x/y/z accelerometer data; distinguishing accelerometer input from programmed sound output

Content

Today, design a motion-recording plan—not a trained system. Build on your previous orientation exploration and example-data discussion; live data collection is still pending.

A class is a category the model will learn to recognize. In supervised learning, a label identifies the intended class of a training example. Choose labels that describe movements rather than the sounds you might eventually want.

Define two deliberate gestures and one idle/background class. The idle/background class represents examples of conditions in which you intend no action, including stillness and selected ordinary movements. Calling a class “no action” does not itself program an output behavior.

Make each gesture definition specific: starting position, movement direction, movement size, and ending position. Changing a sensor's orientation changes how physical acceleration is distributed across its x/y/z axes. Keeping the device's placement and starting orientation consistent reduces one source of variation, making it easier to compare examples of the intended gestures.

Plan variation within those definitions. Varied repetitions provide examples of more than one way to perform a class. Equal planned counts prevent any class from being outnumbered in the plan, but do not guarantee accurate recognition.

Finally, reserve new performances for later evaluation. Evaluating on fresh performances checks recognition on examples that were not used for training. Today you will plan those performances, not record them or test a model.

Activity

Planning workshop: three classes, one consistent setup

Use the board and its already-attached USB cable from the previous lesson. Leave the cable's other end unconnected and the optional battery pack disconnected. Do not repeat connection preparation or claim that live readings are available. Deliver your plan aloud; no additional recording equipment is needed today.

1. Agree on a device-placement rule.

Hold the board gently by its edges, close to the work surface, with the front facing upward and the USB socket pointing away from you. Keep the cable slack. Use the same hand and grip for all rehearsals. If the cable restricts a movement, make the movement smaller rather than pulling the cable.

2. Rehearse two candidate gestures.

- SIDE: From the agreed starting pose, move the board a short distance to the left and back to the starting location, keeping its face upward.
- LIFT: From the same starting pose, lift the board a short distance upward and lower it back, keeping its face upward.

Perform these slowly without tossing, striking, or releasing the board. Agree on a comfortable movement size for each and demonstrate its start and finish. Treat these as candidate definitions, not proof that a model can distinguish them.

3. Define BACKGROUND and its boundaries.

Use three planned background conditions: resting face-up on the work surface, held still in the agreed pose, and tiny grip adjustments without a deliberate SIDE or LIFT movement. Agree on what counts as a tiny adjustment by demonstrating it. If a demonstration resembles a deliberate gesture, revise the definitions before proceeding.

For each class, say: "This class includes _____. It does not include _____."

4. Explain a balanced recording plan.

Use this proposed target for later collection—not a required sample count or a guarantee of success:

- SIDE: 12 examples—4 slower, 4 comfortable-speed, and 4 slightly quicker performances.
- LIFT: 12 examples—the same speed grouping.
- BACKGROUND: 12 examples—4 resting, 4 held still, and 4 with tiny grip adjustments.

Keep every performance gentle and within the agreed movement definition. Keep the placement rule unchanged. Count the planned examples aloud: 12 per class, 36 altogether. These are planned examples, not collected data.

5. Check whether the definitions are usable.

Ask a partner to name the intended class while you demonstrate SIDE, LIFT, and one background condition. Ask the partner to repeat each definition in their own words. Resolve any disagreement about directions, movement size, or class boundaries. Do not call this a model test.

6. Reserve fresh evaluation performances.

Add this commitment to the plan: "After training in a later lesson, we will make three new performances per class for evaluation. Those particular performances will not be training examples." Do not perform

that evaluation set now.

7. Give a handoff report.

State your three labels, each class's inclusion and exclusion rules, your placement rule, the planned counts and variations, and your fresh-evaluation commitment. End with: "Connection and recording remain pending. Today we completed definitions and a recording plan." Stop before collecting examples, training, or programming outputs.

Equipment Used: micro:bit, USB cable

Claims (9) — review these first

[MECHANISTIC] A class is a category the model will learn to recognize.

[MECHANISTIC] In supervised learning, a label identifies the intended class of a training example.

[MECHANISTIC] The idle/background class represents examples of conditions in which you intend no action, including stillness and selected ordinary movements.

[MECHANISTIC] Calling a class "no action" does not itself program an output behavior.

[MECHANISTIC] Changing a sensor's orientation changes how physical acceleration is distributed across its x/y/z axes.

[CAUSAL] Keeping the device's placement and starting orientation consistent reduces one source of variation, making it easier to compare examples of the intended gestures.

[MECHANISTIC] Varied repetitions provide examples of more than one way to perform a class.

[MECHANISTIC] Equal planned counts prevent any class from being outnumbered in the plan, but do not guarantee accurate recognition.

[MECHANISTIC] Evaluating on fresh performances checks recognition on examples that were not used for training.

Assessment

Why does the recording plan keep device placement and starting orientation consistent while allowing modest differences in gesture speed?

- Consistent placement removes the need for a background class.
- Consistent placement reduces one source of variation so the intended gestures are easier to compare. ✓ CORRECT
- Consistent placement guarantees that every future gesture will be recognized.
- Consistent placement automatically programs the no-action response.

(Correct answer justified by claim #5 above)

Lesson 3: Build a Labeled Motion Dataset

Learning Target: Record labeled examples in CreateAI and inspect the dataset for mislabeled, interrupted, or underrepresented motion samples.

Introduces: creating labeled classes in CreateAI; recording accelerometer samples in CreateAI; checking motion samples for label accuracy and recording quality; checking sample balance across classes

Prerequisites: connecting micro:bit via USB; opening micro:bit CreateAI in a supported browser; installing CreateAI connection software on micro:bit and pairing it with the browser; reading x/y/z accelerometer data; defining gesture classes and an idle/background class; using class labels for supervised learning; controlling micro:bit placement and orientation; planning varied and balanced motion recordings; reserving fresh gesture performances for evaluation

Content

Today's focus is dataset quality: deciding whether examples match their labels, capture the intended motion, and represent every planned class.

micro:bit CreateAI trains a machine learning model on accelerometer/motion data, not audio or images. Your SIDE, LIFT, and BACKGROUND definitions from the previous lesson remain the collection plan.

Collection boundary: Without a supplied browser-access device, we cannot complete the planned browser-based recording task with the listed equipment. Connection and recording are still pending from the earlier lessons. Do not assume access to class-creation, recording, sample-review, or editing controls beyond the supplied tool description. Instead, complete the clearly labeled practice audit below. This is preparation for dataset collection, not a claim that you have recorded examples or fully met the recording target.

Use three checks:

- **Label accuracy:** A labeled training example pairs input data with an intended class. If a performance meets the LIFT definition but carries the SIDE label, the label and performance disagree.
- **Recording quality:** An interrupted recording may omit part of the intended gesture, so it should be flagged for review rather than automatically accepted as a complete example.
- **Representation:** Check usable examples in each class and coverage of the variations in your plan. Equal class counts do not establish that all planned variations are represented.

An unusual-looking example is not automatically a bad example: a complete, correctly labeled performance can still differ in speed from other examples. Judge it against the agreed gesture definition rather than rejecting it simply for being different.

Activity

Practice dataset audit — no live recording

Use the class definitions and placement rule you already established. Leave the board and its already-attached USB cable as they are; do not repeat setup, rehearse a new gesture plan, or claim that measurements have been collected. Give your audit aloud.

1. Read this fictional collection report.

These are invented audit notes, not recordings from your board or output from a software interface.

- **SIDE:** 12 reported examples. Eleven are described as complete SIDE performances. One, S07, is labeled SIDE, but the observer reports a complete LIFT performance.
- **LIFT:** 12 reported examples. Ten are described as complete LIFT performances. Two, L03 and L09, ended before the lowering movement was captured.
- **BACKGROUND:** 8 reported examples. Four show resting and four show holding still. None show the planned tiny grip adjustments. All eight are described as complete and correctly labeled.

2. Audit label accuracy.

Name the example with a label-performance disagreement. Explain the disagreement using the existing class definitions. Recommend that its label and original performance be checked before it is accepted. Do not assume that any software offers relabeling or deletion.

3. Audit recording quality.

Name the two interrupted examples. Explain what part of the intended motion is missing. Recommend collecting complete replacements when recording becomes available; do not perform or record replacements now.

4. Recount provisionally usable examples.

For this exercise, set aside S07, L03, and L09. Count only the remaining examples described in the report as complete and correctly labeled. State the count for each class and identify the smallest group.

5. Check coverage, not just totals.

Compare the BACKGROUND report with your existing plan. Which background condition is absent? Explain why collecting more resting examples alone would not fill that gap.

6. Challenge an overstrict audit rule.

Consider this proposed rule: "Reject every example that is slower than the others." Explain why that rule conflicts with your existing plan for varied speeds. Replace it with a check for a complete performance that still meets its class definition.

7. Give a handoff report.

Include the suspect example IDs, your provisional usable counts, the missing background condition, and your recommended collection priorities. Close with: "We audited a fictional dataset. Live connection, labeled recording, and inspection of actual samples remain pending." Stop before training, testing predictions, or programming sounds.

Claims (10) — review these first

[HARD_FACT] (source: [micro:bit Educational Foundation — Official CreateAI product page and user guide; motion-based machine learning, consistent with the supplied capabilities summary.](#)) micro:bit CreateAI trains a machine learning model on accelerometer/motion data, not audio or images.

[CAUSAL] Without a supplied browser-access device, we cannot complete the planned browser-based recording task with the listed equipment.

[MECHANISTIC] A labeled training example pairs input data with an intended class.

[MECHANISTIC] If a performance meets the LIFT definition but carries the SIDE label, the label and performance disagree.

[CAUSAL] An interrupted recording may omit part of the intended gesture, so it should be flagged for review rather than automatically accepted as a complete example.

[MECHANISTIC] Equal class counts do not establish that all planned variations are represented.

[MECHANISTIC] An unusual-looking example is not automatically a bad example: a complete, correctly labeled performance can still differ in speed from other examples.

[PEDAGOGICAL] SIDE: 12 reported examples. Eleven are described as complete SIDE performances. One, S07, is labeled SIDE, but the observer reports a complete LIFT performance.

[PEDAGOGICAL] LIFT: 12 reported examples. Ten are described as complete LIFT performances. Two, L03 and L09, ended before the lowering movement was captured.

[PEDAGOGICAL] BACKGROUND: 8 reported examples. Four show resting and four show holding still. None show the planned tiny grip adjustments. All eight are described as complete and correctly labeled.

Assessment

A team reaches 12 examples in each class, but its BACKGROUND examples contain only resting—even though the plan also includes holding still and tiny grip adjustments. Which conclusion is justified?

- The dataset is ready because equal counts guarantee complete coverage.
- The class counts are balanced, but planned background variations are still missing. ✓ CORRECT
- The resting examples must all be mislabeled.
- The team should remove the BACKGROUND class.

(Correct answer justified by claim #5 above)

Lesson 4: Train a Model and Examine Its Mistakes

Learning Target: Train a CreateAI model, compare its predictions with intended labels on fresh performances, and identify commonly confused classes.

Introduces: training a motion-classification model in CreateAI; evaluating live model predictions with fresh performances; tabulating intended labels against predicted labels; distinguishing learned classification from explicit programming rules

Prerequisites: opening micro:bit CreateAI in a supported browser; installing CreateAI connection software on micro:bit and pairing it with the browser; creating labeled classes in CreateAI; recording accelerometer samples in CreateAI; checking motion samples for label accuracy and recording quality; checking sample balance across classes; controlling micro:bit placement and orientation; reserving fresh gesture performances for evaluation

Content

Today's boundary

Continue from your fictional dataset audit—not from a collected dataset. With connection and actual recording still pending, and no browser-access device listed, do not claim to train or evaluate a real model today. Instead, practice interpreting fictional predictions. For this lesson, use the supplied prediction records rather than assume an unlisted live-testing feature. The live portion of the learning target remains pending.

Training versus programming

micro:bit CreateAI trains a machine-learning model using accelerometer/motion data, not audio or images. Training a classifier uses labeled examples to learn patterns for assigning input to classes. An explicitly programmed classification rule instead specifies the decision conditions directly, such as “if a measured value exceeds this threshold, choose this class.” Learning from examples does not guarantee that every new performance will receive its intended label.

Reading prediction results

The intended label is the class the evaluator meant to perform; the predicted label is the class assigned by the model. Fresh evaluation performances are kept out of training so that the comparison checks performances the model was not trained on.

In an intended-versus-predicted table, rows represent intended labels and columns represent predicted labels. Counts on the matching-label diagonal represent correct classifications; counts outside that diagonal represent mismatches. The largest off-diagonal count identifies the most frequent intended-to-predicted confusion in that evaluation set. A small evaluation set does not establish how reliably a model will classify all future performances.

Keep classification separate from the project's later sound-response work. Today, stop at interpreting predictions and identifying confusion.

Activity

Prediction-reading workshop — fictional results only

Leave the existing board and cable setup unchanged. Work aloud with a partner using the records below; no new equipment or software interaction is required.

1. Establish the status.

Say: “Our earlier work produced class definitions, a recording plan, and a fictional dataset audit. These prediction records are another practice example, not output from our board.” Do not treat the audit notes as a trainable dataset.

2. Explain what training would do.

Using SIDE, LIFT, and BACKGROUND, explain the difference between learning from labeled motion examples and writing explicit conditions for choosing a label. Do not create a new dataset, invent a software control, or program a response.

3. Compare intended and predicted labels.

Imagine a hypothetical model receiving three fresh performances per class. These invented records are not evidence about your gestures or a real model:

Trial | Intended | Predicted

1 | SIDE | SIDE

2 | SIDE | LIFT

3 | SIDE | SIDE

4 | LIFT | SIDE

5 | LIFT | LIFT

6 | LIFT | SIDE

7 | BACKGROUND | BACKGROUND

8 | BACKGROUND | BACKGROUND

9 | BACKGROUND | BACKGROUND

For each trial, say “match” or “mismatch.” For every mismatch, name both labels in order: “intended ___, predicted ___.”

4. Build the comparison table aloud.

Use rows in this order: SIDE, LIFT, BACKGROUND. Use columns in that same order. One partner reads the trial records while the other reports each row's three counts:

Intended SIDE: predicted SIDE ___, LIFT ___, BACKGROUND ___.
Intended LIFT: predicted SIDE ___, LIFT ___, BACKGROUND ___.
Intended BACKGROUND: predicted SIDE ___, LIFT ___, BACKGROUND ___.

Switch roles and check each row against the records. Check that each row totals three and all cells together total nine.

5. Identify the confusion.

Which off-diagonal cell has the largest count? State the confusion with its direction and count. Then identify the two classes involved. Does the reverse-direction confusion have the same count? Support your answer with trial numbers.

6. Separate evidence from overclaiming.

Discuss this statement: “All three BACKGROUND trials matched, so the model will never confuse background movement.” Explain why the conclusion goes beyond these fictional results. Do not test changed users, orientations, or sound responses today.

7. Give a handoff report.

Report the fictional table's most frequent confusion, the trial numbers supporting it, and why fresh performances should be separate from training examples. Close with: “We practiced evaluating fictional predictions. Actual recording, training, and live evaluation remain pending.” Stop before programming sounds or transferring a program.

Claims (10) — review these first

[HARD_FACT] (source: [micro:bit Educational Foundation — Official CreateAI product page and user guide; motion-based model training, consistent with the supplied capability summary](#).) micro:bit CreateAI trains a machine-learning model using accelerometer/motion data, not audio or images.

[MECHANISTIC] Training a classifier uses labeled examples to learn patterns for assigning input to classes.

[MECHANISTIC] An explicitly programmed classification rule instead specifies the decision conditions directly, such as “if a measured value exceeds this threshold, choose this class.”

[MECHANISTIC] Learning from examples does not guarantee that every new performance will receive its intended label.

[MECHANISTIC] The intended label is the class the evaluator meant to perform; the predicted label is the class assigned by the model.

[CAUSAL] Fresh evaluation performances are kept out of training so that the comparison checks performances the model was not trained on.

[MECHANISTIC] In an intended-versus-predicted table, rows represent intended labels and columns represent predicted labels.

[MECHANISTIC] Counts on the matching-label diagonal represent correct classifications; counts outside that diagonal represent mismatches.

[MECHANISTIC] The largest off-diagonal count identifies the most frequent intended-to-predicted confusion in that evaluation set.

[MECHANISTIC] A small evaluation set does not establish how reliably a model will classify all future performances.

Assessment

Why should evaluation use fresh performances that were not included in training?

- To check classification on performances the model was not trained on. ✓ CORRECT
- To guarantee that every predicted label will be correct.
- To replace learned patterns with explicitly programmed rules.
- To make the intended label automatically become the predicted label.

(Correct answer justified by claim #5 above)

Lesson 5: Program Gesture-to-Sound Responses

Learning Target: Open the trained model in MakeCode and program distinct sounds for the two gestures, with no sound for the idle/background class and a pause between responses.

Introduces: opening the trained CreateAI model in MakeCode; using MakeCode blocks for recognized-motion events; mapping gesture classes to built-in speaker sounds; programming no sound for the idle/background class; adding a pause to limit repeated sound responses

Prerequisites: checking micro:bit V2 compatibility and locating its built-in speaker; distinguishing accelerometer input from programmed sound output; defining gesture classes and an idle/background class; training a motion-classification model in CreateAI; distinguishing learned classification from explicit programming rules

Content

Today's focus is the response rule: what should happen after a motion label is recognized?

micro:bit CreateAI can trigger a programmed sound output via MakeCode in response to recognized motion. That is the supported output capability for this project.

Scope limitation: keep actual model opening and block editing marked pending. Do not treat the previous lesson's fictional predictions as a trained model. The supplied resources do not specify a USB host/access device or document model-opening controls, recognized-motion blocks, sound choices, or pause blocks. Omit those software steps rather than inventing them. Today's deliverable is a platform-neutral response specification, not an executable or transferred program.

A response mapping associates a recognized label with a programmed action. Use the existing SIDE, LIFT, and BACKGROUND labels; do not redefine or retrain them.

For our specification, choose two distinct intended sounds, called A and B. Assign A to SIDE and B to LIFT. Assign no new sound to BACKGROUND. Here, "no new sound" does not mean cancelling a sound that has already started.

A shared waiting rule prevents either gesture from starting another sound while the response system is busy. Merely choosing different sounds does not impose a waiting period between responses. Specify one shared ready/busy state for both gesture responses, with a one-second pause after a sound finishes. This is an abstract design requirement, not a claim about available software blocks.

Activity

Response-rule workshop — specification only

Leave the existing board and cable setup unchanged. Work aloud with a partner; do not connect, train, open an editor, or transfer a program.

1. Specify the mapping.

Complete these rules using your existing labels:

- SIDE → start sound A.
- LIFT → start sound B.

- BACKGROUND → start no sound.

Describe how you would like A and B to differ, such as a higher versus lower pitch. Treat these as requested sounds, not confirmed menu choices or sounds produced today.

2. Add one shared waiting rule.

Read this platform-neutral specification aloud:

- Begin in READY state.
- When a label arrives, first check the state.
- If BUSY, ignore that label for sound-start purposes; do not save it for later playback.
- If READY and the label is BACKGROUND, do nothing and remain READY.
- If READY and the label is SIDE or LIFT, change to BUSY before starting its assigned sound.
- After that sound finishes, wait one second, then return to READY.

These are proposed response instructions, not software block names.

3. Trace the instructions without making sounds.

One partner reads each event below. The other reports the state and whether a sound should start:

- The system is READY; BACKGROUND arrives.
- SIDE arrives while the system is still READY.
- LIFT arrives while sound A is playing.
- SIDE arrives during the one-second pause after sound A finishes.
- The pause ends; then LIFT arrives.

For each decision, point to the rule that determines it. Treat these as invented label events, not model predictions or device tests.

4. Review the response design.

Check that your specification includes two distinct intended sounds, no sound-start action for BACKGROUND, and one waiting rule shared by both gestures. Explain whether a second gesture should bypass the wait just because its label differs from the first.

5. Give a handoff report.

State the mapping and waiting rule. Close with: “We completed an abstract response specification. Actual model opening, block programming, and execution remain pending.” Stop before transferring anything or verifying sounds on the device.

Claims (5) — review these first

[HARD_FACT] (source: micro:bit Educational Foundation — Official CreateAI product page and user guide; movement recognition and programmed sound responses through MakeCode.) micro:bit CreateAI can trigger a programmed sound output via MakeCode in response to recognized motion.

[MECHANISTIC] A response mapping associates a recognized label with a programmed action.

[MECHANISTIC] Here, “no new sound” does not mean cancelling a sound that has already started.

[MECHANISTIC] A shared waiting rule prevents either gesture from starting another sound while the response system is busy.

[MECHANISTIC] Merely choosing different sounds does not impose a waiting period between responses.

Assessment

In this specification, SIDE starts sound A. LIFT arrives during the shared one-second pause after sound A finishes. What should happen?

- Sound B starts immediately because LIFT is a different label.
- No new sound starts; the system is still busy. ✓ CORRECT
- Sound A starts again because SIDE was recognized first.
- Sound B is saved to play automatically when the pause ends.

(Correct answer justified by claim #3 above)

Lesson 6: Run the Gesture-to-Sound System on the Device

Learning Target: Transfer the model-enabled program to micro:bit and verify each gesture-to-sound mapping and the silent idle response without browser-based inference.

Introduces: transferring the model-enabled MakeCode program to micro:bit; running model inference and sound responses on micro:bit; verifying end-to-end gesture-to-sound mappings; distinguishing recognition errors from sound-programming errors

Prerequisites: connecting micro:bit via USB; controlling micro:bit placement and orientation; evaluating live model predictions with fresh performances; opening the trained CreateAI model in MakeCode; using MakeCode blocks for recognized-motion events; mapping gesture classes to built-in speaker sounds; programming no sound for the idle/background class; adding a pause to limit repeated sound responses

Content

Today's goal is an end-to-end check: does each intended gesture lead to its assigned sound, and does background activity remain silent?

Implementation boundary: the earlier workshops ended with an abstract response specification, not a trained model or executable program. A USB host/access device is not included in the supplied equipment, and transfer procedures and browser-independent inference are not established by the supplied software capabilities. Do not attempt an invented transfer procedure or report successful device execution. For this lesson, complete a verification-readiness and diagnostic exercise; the practical learning target remains pending.

Use the existing specification: SIDE → sound A; LIFT → sound B; BACKGROUND → no new sound. Keep the shared waiting rule unchanged.

An end-to-end check compares the intended gesture, the predicted class, the programmed sound action, and the observed sound. A recognition error occurs when the predicted class differs from the intended class. A sound-mapping error occurs when the programmed sound action differs from the action assigned to the predicted class, provided the response rules allow an action at that time. A wrong prediction can therefore produce the wrong sound for the intended gesture even when the sound mapping works correctly.

Silence alone does not establish correct background recognition: a waiting rule or an output failure can also prevent a new sound. For a silent-idle check, begin after any earlier sound and waiting period have ended.

Browser-independent inference means that the device, rather than a browser, computes the predicted class. Hearing a device make a sound does not by itself establish where the prediction was computed.

Activity

Verification-readiness workshop — fictional evidence, not device execution

Keep the board and its already-attached cable unchanged. Work aloud with a partner. Do not repeat connection preparation, dataset planning, training, or response-rule design.

1. Check the handoff honestly.

Name what is available from earlier lessons: gesture definitions, a recording plan, practice audits, fictional prediction results, and an abstract response specification. Identify the missing evidence needed for a real device check: an actual trained model, an implemented program, a supported transfer method, and confirmation of where inference runs. Mark actual transfer and device verification as pending.

2. State the acceptance checks.

Using the existing mapping, say the expected result for one SIDE performance, one LIFT performance, and each previously defined background condition: resting, held still, and tiny grip adjustments. Keep the original placement, orientation, user, and gesture definitions. For each check, require the previous sound and waiting period to have ended. Do not run these checks today or change the rules.

3. Audit these fictional end-to-end records.

For this exercise only, assume every trial begins READY, with no earlier sound still playing. “Command” means the sound-start action requested by the hypothetical program. These records are invented, not software logs or measurements from your board.

Trial | Intended class | Predicted class | Command | Heard

1 | SIDE | SIDE | A | A

2 | LIFT | LIFT | B | B

3 | BACKGROUND | BACKGROUND | None | Silence

4 | SIDE | LIFT | B | B

5 | LIFT | LIFT | A | A

6 | BACKGROUND | BACKGROUND | B | B

For each row, answer three separate questions:

- Does the prediction match the intention?
- Does the command match the existing rule for that prediction?
- Does the heard result match the command?

4. Locate the failed comparison.

Identify the row showing a recognition error with a correct sound mapping. Identify the two rows showing correct recognition but incorrect sound mapping. Explain each decision by naming the two columns you compared. Do not retrain or redesign anything.

5. Check the silent-idle evidence.

Compare trials 3 and 6. Which supports the silent-background requirement for that fictional trial? Which violates it? Now imagine a record that says only “BACKGROUND intended; silence heard.” Explain what information is missing before you can judge recognition and response logic separately.

6. Examine a claim about independent operation.

Consider: “I heard a sound from the board while a browser was open, so inference must have happened on the board.” Explain why this observation is insufficient. State that an actual verification would need a supported procedure and evidence that predictions continue without browser-based inference. Do not invent controls, disconnect equipment, or claim this test has occurred.

7. Give a handoff report.

Report your acceptance checks, the recognition-error trial, the sound-mapping-error trials, and the missing evidence for independent operation. End with: “We practiced interpreting end-to-end evidence. Transfer, browser-independent inference, and actual gesture-to-sound verification remain uncompleted.” Stop before testing changed users, speeds, or orientations.

Equipment Used: micro:bit, USB cable

Claims (7) — review these first

[MECHANISTIC] An end-to-end check compares the intended gesture, the predicted class, the programmed sound action, and the observed sound.

[MECHANISTIC] A recognition error occurs when the predicted class differs from the intended class.

[MECHANISTIC] A sound-mapping error occurs when the programmed sound action differs from the action assigned to the predicted class, provided the response rules allow an action at that time.

[CAUSAL] A wrong prediction can therefore produce the wrong sound for the intended gesture even when the sound mapping works correctly.

[MECHANISTIC] Silence alone does not establish correct background recognition: a waiting rule or an output failure can also prevent a new sound.

[MECHANISTIC] Browser-independent inference means that the device, rather than a browser, computes the predicted class.

[MECHANISTIC] Hearing a device make a sound does not by itself establish where the prediction was computed.

Assessment

A fictional trial begins READY. The intended gesture is SIDE, the predicted class is LIFT, and the program correctly commands and plays sound B for LIFT. What does this evidence show?

- A recognition error, with the sound mapping working correctly. ✓ CORRECT
- Correct recognition, with a sound-mapping error.
- Both correct recognition and correct end-to-end behavior.
- Proof that inference ran without a browser.

(Correct answer justified by claim #3 above)

Lesson 7: Audit Reliability Beyond the Original Gestures

Learning Target: Compare unintended and missed sound responses across changed users, gesture speeds, and device orientations, then recommend a specific improvement supported by recorded evidence.

Introduces: designing a robustness audit that changes one condition at a time; measuring unintended activations and missed responses; connecting performance changes to training-data coverage; documenting operating limits and evidence-based improvement priorities

Prerequisites: controlling micro:bit placement and orientation; planning varied and balanced motion recordings; tabulating intended labels against predicted labels; running model inference and sound responses on micro:bit; verifying end-to-end gesture-to-sound mappings; distinguishing recognition errors from sound-programming errors

Content

Today, audit fictional records rather than test a live device. Actual training, programming, transfer, and device verification remain pending from the earlier workshops; do not treat their practice records as completed deployment.

A robustness audit compares performance under a reference condition with performance under deliberately changed conditions. Change one condition at a time: user, gesture speed, or starting orientation. Keeping other conditions fixed makes a performance difference easier to associate with the condition being investigated.

Use the existing response specification: SIDE should produce A, LIFT should produce B, and BACKGROUND should produce silence. For this audit, count three outcomes separately:

- Missed response: a deliberate SIDE or LIFT trial produces silence during its observation window.
- Unintended activation: a BACKGROUND trial produces any sound during its observation window.
- Wrong-sound response: a deliberate gesture produces the other gesture's sound rather than its assigned sound.

The missed-response rate is silent gesture trials divided by all deliberate gesture trials. The unintended-activation rate is background trials with sound divided by all background trials. These rates use different denominators.

Sound-only records describe response outcomes but cannot, by themselves, distinguish recognition errors from response-programming errors. Differences across conditions can motivate a training-coverage hypothesis, but they do not prove that missing training examples caused the errors. A limited audit supports conclusions about the tested conditions, not a guarantee for every user or movement.

Activity

Fictional robustness audit — work aloud with a partner

Do not change the existing board-and-cable setup or attempt unfinished deployment steps. Use the supplied records as your evidence; no live measurements or additional recording equipment are needed.

1. Check the audit design.

For this fictional scenario, use these conditions:

- Reference: original user, comfortable speed, original face-up orientation.
- Changed user: second user, comfortable speed, original orientation.
- Changed speed: original user, slower deliberate gestures, original orientation.
- Changed orientation: original user, comfortable speed, starting with the board on an edge instead of face-up.

Assume movement size, class definitions, trial counts, and observation-window length are held constant. Background trials use the same sequence of held-still and tiny-grip-adjustment tasks in every condition. Each trial begins READY after any previous sound and waiting period have ended. These are assumptions of the fictional records, not verified properties of your setup.

Identify the single changed condition in each comparison. Explain why changing both user and orientation together would make interpretation harder.

2. Read the recorded outcomes.

Each symbol below represents one separate trial. A and B mean the corresponding sound was heard; “–” means silence. Each group contains four trials of the indicated intended class. These are invented observer records, not software logs.

Condition | SIDE outcomes | LIFT outcomes | BACKGROUND outcomes

Reference | A A A A | B B B B | – – – –

Changed user | A A – – | B B B B | A – – –

Changed speed | A A – – | B B B – | – – – –

Changed orientation | B A A – | B B B B | A B – A

One partner reads each row. The other identifies missed responses, unintended activations, and wrong-sound responses. Switch roles and check the classifications. Do not count a wrong sound as silence.

3. Report comparable rates.

For each condition, complete this spoken summary:

“Missed responses: ____ out of 8 deliberate gestures. Unintended activations: ____ out of 4 background trials. Wrong-sound responses: ____ out of 8 deliberate gestures.”

Compare every changed condition with the reference. Which has the most missed responses? Which has the most unintended activations? Cite the recorded outcomes, not just a general impression that one condition is ‘worse.’

4. Investigate coverage without claiming a cause.

Recall that your earlier recording plan included several gesture speeds but kept one placement rule. It was a plan, not a collected dataset.

Discuss two questions:

- Which changed condition tests variation that the original orientation rule would exclude?

- Does planning slower recordings prove that an eventual trained model would handle slower gestures reliably?

Propose a coverage hypothesis using: “One possible explanation is _____. To check it, we would need _____.” Include actual training records and separate recognition-versus-response evidence among the evidence you would request. Do not invent that evidence.

5. Recommend one improvement priority.

Choose either reducing missed deliberate responses or reducing unintended background sounds as your priority. Give one specific recommendation for future work, supported by counts from the audit.

For example, you might recommend collecting complete SIDE, LIFT, and BACKGROUND examples in the edge-start orientation if reducing unintended sounds is your priority. Present this as a candidate improvement to investigate, not a proven fix. Do not collect examples, retrain, or change the response specification today.

6. Deliver an operating-limits report.

Give a short report containing:

- The reference and the one-condition-at-a-time comparisons.
- Two numerical comparisons supporting your chosen priority.
- One proposed improvement and the evidence still needed to justify it.
- A boundary statement: “These fictional results do not establish our device’s operating limits. An actual audit would still need fresh trials after implementation, including conditions beyond this small example.”

Claims (12) — review these first

[PEDAGOGICAL] Today, audit fictional records rather than test a live device.

[MECHANISTIC] A robustness audit compares performance under a reference condition with performance under deliberately changed conditions.

[CAUSAL] Keeping other conditions fixed makes a performance difference easier to associate with the condition being investigated.

[MECHANISTIC] Missed response: a deliberate SIDE or LIFT trial produces silence during its observation window.

[MECHANISTIC] Unintended activation: a BACKGROUND trial produces any sound during its observation window.

[MECHANISTIC] Wrong-sound response: a deliberate gesture produces the other gesture’s sound rather than its assigned sound.

[MECHANISTIC] The missed-response rate is silent gesture trials divided by all deliberate gesture trials.

[MECHANISTIC] The unintended-activation rate is background trials with sound divided by all background trials.

[MECHANISTIC] These rates use different denominators.

[MECHANISTIC] Sound-only records describe response outcomes but cannot, by themselves, distinguish recognition errors from response-programming errors.

[MECHANISTIC] Differences across conditions can motivate a training-coverage hypothesis, but they do not prove that missing training examples caused the errors.

[MECHANISTIC] A limited audit supports conclusions about the tested conditions, not a guarantee for every user or movement.

Assessment

A team changes only the starting orientation and observes more unintended sounds. Which conclusion is best supported?

- The change proves that the training set lacked edge-orientation examples.
- Orientation is worth investigating, but training coverage and response logic need further evidence before naming the cause. ✓ CORRECT
- The team should change user, speed, and orientation together to isolate the cause.
- Any sound during a deliberate gesture must count as a missed response.

(Correct answer justified by claim #10 above)